

Weft Sandbox-Constrained Roadmap

Reframed plan: full development, testing, and reporting inside the Linux sandbox. Real-device verification deferred.

CONSTRAINT ACKNOWLEDGED

The entire project is developed, tested, and reported in-sandbox. Real-device verification is Phase 6+, gated on the constraint lifting. The canonical artifact (the litmus suite) is fully runnable here in three languages (C, Rust, TypeScript). The platform ports (Kotlin, Swift, Dart) are source-only until real devices are available.

0. The constraint

0.1 What the sandbox can compile and run

Toolchain	Version	Available	Role in project
GCC (C/C++)	14.2.0	YES	Kernel + litmus runner + tools
Rust (rustc + cargo)	1.98.1	YES (installed via rustup)	Kernel + litmus runner + tools
Node.js + npm	24.19.0 / 11.17.0	YES	TypeScript kernel + litmus runner + web Heddles
Python	3.12.14	YES	Benchmark harness + result pipeline + structural validators
Bash + Unix	(n/a)	YES	CI glue, Makefiles
OpenJDK (with javac)	(n/a)	NO (JRE only)	Kotlin/Android source-only
Swift	(n/a)	NO	Swift/iOS source-only
Dart / Flutter	(n/a)	NO	Dart/Flutter source-only

0.2 What this means for the project

- **The kernel can be implemented, litmus-tested, and benchmarked in three languages: C, Rust, TypeScript.** All three toolchains are live. The protocol can be proven sound across all three on x86_64 Linux.
- **The platform ports (Kotlin/Android, Swift/iOS, Dart/Flutter) are source-only.** Source files are written and structurally validated by a Python parser, but cannot be compiled or run in-sandbox. Marked "pending real-device verification" and delivered as Phase 6+ artifacts.
- **The canonical artifact (the litmus suite) does not require a real device.** It is a YAML catalog plus per-language runners. Runners run on whatever hardware the language supports.
- **The moat (the benchmark suite) does not require a real device for the protocol-proof numbers.** It does require real devices for the platform numbers (Pixel 7a vs Galaxy S24, etc.). Sandbox numbers are labeled x86_64-sandbox.
- **Real-device numbers replace sandbox numbers when the constraint lifts.** Until then, the spec, docs, and whitepaper carry sandbox numbers with explicit labels.

0.3 What changes vs the canonical ROADMAP

Canonical phase	Reframed phase	What changes
Phase 0 (Kotlin spike)	Phase 0 (C + Rust + TS kernels + litmus)	Reference language is C, not Kotlin; three implementations instead of one
Phase 1 (Android v0.1)	Phase 4 (source-only) + Phase 6+ (real device)	Split: source written and validated in-sandbox; compilation and on-device tests deferred
Phase 2 (Web v0.1)	Phase 0e/0f (TS kernel + litmus) + Phase 6+ (browser publish)	Kernel and litmus in-sandbox; browser deployment deferred
Phase 3 (Benchmark launch)	Phase 1 (Python harness) + Phase 5 (sandbox release)	Harness and site mockup in-sandbox; public weft.dev launch deferred
Phase 4 (Whitepaper)	Phase 3	In-sandbox, uses sandbox numbers (clearly labeled)
Phase 5 (SwiftUI)	Phase 4 (source-only) + Phase 6+	Source written; compilation deferred
Phase 6 (Flutter)	Phase 4 (conditional, source-only) + Phase 6+	Same
Phase 7 (React Native)	Phase 4 (source-only, last) + Phase 6+	Same
Phase 8 (Pro tier)	Out of scope	Separate repo per charter clause 4

0.4 What does not change

- **The Four Laws.** Reader never blocked, zero is a contract, mechanism not policy, honesty is a feature. The honesty law now includes "sandbox numbers are labeled sandbox numbers."
- **The litmus suite is the canonical core.** Three languages run it; the project's claim to "Weft" is passing it, not lineage.
- **The frame envelope is Tier 0 frozen.** The 16-byte header is implemented once per language and never changes shape.
- **Writer revocation (I6) is implemented from day one.** The use-after-free guard across FFI is non-negotiable; it costs one relaxed load per publish.
- **Sequential, benchmark-gated phases.** A phase ends when its success criterion is met, not when its calendar slot does. Slippage is published.

Phase 0 — Corrected kernel + L1–L8 litmus suite, in three languages

The single most important phase. This is the only thing that can kill the project, and the founding spec's version of it (the two-variable protocol) was formally unsound. De-risk it first, adversarially, in three languages, or nothing after this matters.

0a — C kernel (corrected Triad Protocol)

Re-implement the protocol from RFC 0001 §4, not the withdrawn two-variable design from the founding spec §5.

- **Single shared atomic** (latest), writer-private `w_work`, reader-private `r_work`.
- **Publish via `latest.swap(w_work, AcqRelease)`** — one RMW, one ownership transfer.
- **Claim via `latest.swap(r_work, AcqRelease)`** — same primitive, same semantics.
- **Frame envelope** (Tier 0 frozen): 16-byte header before payload, every buffer, every language.
- **Writer revocation token** (revoked: `AtomicBool`, epoch: `u32`): checked once per publish; release sets revoked (`Release`) before free; free deferred until writer quiescence.
- **Zero dependencies** — pure C11, `pthread`, `stdatomic`, `posix_memalign`.

LOC budget: ~600 (kernel + envelope + I6).

0b — C litmus runner

The runner executes the L1–L8 catalog against the kernel. The catalog itself is language-agnostic (Phase 0g); the runner is the only platform-specific code.

Test	Adversarial condition	Verdict
L1-tear	Reader hold stretched 5/10/50 ms between swap and read; writer at 2× display rate	Every claimed payload consistent with envelope seq; zero partial frames
L2-writer-steps	Instrumented publish under reader holds swept 0–100 ms	Publish step count ≤ hard bound regardless of reader
L3-reader-steps	Instrumented claim under writer storms (4× rate)	Claim step count ≤ hard bound; never retries, never fails
L4-freshness	Writer at 4× reader rate	Claimed seq ≥ newest published at claim time, every frame
L5-progress	Reader hold swept 0–100 ms; reader suspended entirely	Writer throughput flat (±noise) in every config
L6-ownership	Canary word per buffer; randomized interleavings	Non-owner mutation of canary fails
L7-revocation	<code>release()</code> under spinning native writer; freed pages poisoned	Zero writes to poisoned pages; <code>DROPPED_REVOKED</code> within one publish
L8-envelope	Round-trip; unknown trailing fields; version negotiation	Byte-identical round-trip; unknown fields ignored; triad-1 + triad-2 coexist

LOC budget: ~1,200 (8 tests × harness code).

0c — Rust kernel

Same protocol, same envelope, same I6 contract. Rust's `std::sync::atomic` maps cleanly to C11 `<stdatomic.h>`; `alloc::alloc` with `Layout` provides the off-heap buffer. No unsafe in the public API; every unsafe block has a SAFETY: comment citing the RFC section.

LOC budget: ~600.

0d — Rust litmus runner

Same catalog, Rust runner. Validates that the protocol's soundness is not a C-specific artifact.

LOC budget: ~1,200.

0e — TypeScript kernel

The web port. Uses `SharedArrayBuffer` (Node 24 supports it without COOP/COEP for in-process Workers) + `Atomics` for the single atomic. Frame envelope serialized as a `DataView` over the SAB.

LOC budget: ~700 (slightly larger due to SAB setup and Worker message protocol).

0f — TypeScript litmus runner

Same catalog, TypeScript runner. Uses `worker_threads` for the writer and reader threads. Validates the protocol on the web memory model (sequentially consistent for `Atomics` — stronger than C's relaxed ordering).

LOC budget: ~1,400 (Worker setup + message passing overhead).

0g — Litmus catalog (language-agnostic)

The catalog is the project's canonical specification. YAML file listing each test's scenario, adversarial parameters, and verdict. Runners in C, Rust, and TS all parse the same catalog and execute it.

LOC budget: ~400 (YAML spec + Python catalog validator).

Phase 0 success criterion

- **L1–L8 green in C, Rust, and TypeScript**, each in debug and release builds.
- L1 tears = 0 under all adversarial holds (5/10/50 ms).
- L2/L3 step bounds hold on release builds.
- L7 zero writes to poisoned pages under concurrent release.
- Cross-language consistency: each test passes in all three languages, or the protocol is wrong.
- RFC 0001 flips to Status: Accepted.

Deliverables

- `core/c/` — C kernel + litmus runner
- `core/rust/` — Rust kernel + litmus runner
- `core/ts/` — TypeScript kernel + litmus runner
- `litmus/catalog.yaml` — language-agnostic test catalog
- `litmus/REPORT.md` — Phase 0 results, 24 green-cell combinations (8 tests × 3 languages)
- PDF: `Weft-Phase0-Litmus-Report.pdf`

Phase 1 — Python benchmark harness + result pipeline

The benchmark suite is the moat. The harness is the engine that produces the moat's numbers.

1a — Python harness wrapping the C and Rust kernels

Wraps the C kernel (via ctypes) and the Rust kernel (via cdylib + ctypes) as interchangeable backends. Harness selects the backend at runtime; workload code is identical regardless of backend.

1b — Five workloads (W1–W5)

Each workload is a config + a draw routine. The draw routine is the same shared drawBar-class function across all four implementations (A/B/C/D) — the fairness pin.

Workload	Hot state	Bounds
W1 Audio visualizer	1024-float PCM @ 60/120 Hz	Fixed
W2 Particle field	500 particles × 6-DOF RK4 @ 120 Hz	Fixed
W3 Spectrogram / heatmap	256×64 float matrix @ 60 Hz	Fixed
W4 Data grid	10k rows × 20 cols, live updates	Fixed-cell subset
W5 Order book	1000 levels × 10 fields, 60 Hz L2 feed	Fixed

1c — Four implementations (A/B/C/D)

- **A — Reactive naive:** hot state in a Python list (closest analog to MutableState), redrawn every frame.
- **B — Best practice:** pooled array.array, no per-frame allocation, deferred draw (memcpy into pre-allocated render buffer).
- **C — Weft:** the C kernel via ctypes.
- **D — Hand-rolled:** a hand-coded triple-buffer in Python with the same envelope and I6 contract.

1d — Metrics

- P50 / P99 / P100 FPS (**P99 is the headline**)
- Frame allocation rate (bytes/frame, asserted at 0 for C and D)
- GC pause count + total ms (via gc.get_stats())
- CPU% (via psutil)
- Cold start overhead
- Thermal sustained (30-minute run, FPS decay curve)

1e — Fairness pin verification

The harness verifies mechanically that A/B/C/D call the same drawBar function with the same values per frame. Draw-call count and element count are recorded per frame and asserted equal across implementations. A result bundle without fairness-pin verification is not a result.

1f — Result bundle format (JSON)

Every run emits a signed result bundle: raw frame timestamps, alloc counters, gc logs, device state (sandbox: x86_64-linux, OS version, CPU model, RAM), harness version, fairness-pin verification.

1g — Static site generator

Python script that reads result bundles from bench/results/ and emits a static HTML+CSS site (no JS framework, no backend) at bench/site/. The site is the future weft.dev/benchmarks mockup, fully rendered in-sandbox.

Phase 1 success criterion

- All five workloads × four implementations × two backends (C and Rust) produce result bundles.
- Fairness pin verified in every bundle.
- C and D measure 0 B/frame (asserted; fails the run otherwise).
- Static site renders all bundles.
- P99 FPS for C and D within 5% of each other (Weft = hand-rolled; if not, the library costs something).

Deliverables

- bench/harness/ — Python harness
- bench/workloads/ — W1–W5 configs + draw routines
- bench/results/ — result bundles (JSON)
- bench/site/ — static HTML site
- PDF: Weft-Phase1-Benchmark-Report.pdf

Phase 2 — Tools (weft-probe, weft-record)

Debug tooling that lives outside the hot path. Both are pure C and Rust; both run in-sandbox.

2a — weft-probe

Inspects a running Weft's state: dumps the envelope, the latest index, the w_work and r_work indices, telemetry counters, the canary words. Connects to a Weft via a shared-memory name (no FFI; reads the buffer header directly).

LOC budget: ~400 (C) + ~400 (Rust).

2b — weft-record

Captures frames for debug replay. Writes a .weftrec file (envelope + seq + payload per frame, binary). The file format is the project's first non-Tier-0 artifact; it lives in tools/ and is versioned with the same semver discipline as the kernel public API.

LOC budget: ~600 (C) + ~600 (Rust).

Phase 2 success criterion

- weft-probe correctly dumps state from a running C-kernel Weft and a Rust-kernel Weft.
- weft-record captures a 30-second run; replay byte-identical.
- Both tools' file formats documented with round-trip tests.

Deliverables

- tools/weft-probe/ — C and Rust implementations
- tools/weft-record/ — C and Rust implementations
- tools/FORMATS.md — file format specs
- PDF: Weft-Phase2-Tools-Report.pdf

Phase 3 — Whitepaper (synthesized from real measurements)

The citable artifact. Every number in it is measured in-sandbox, labeled as x86_64-sandbox. The whitepaper supersedes the founding spec PDF; the founding spec is explicitly marked STATUS: SUPERSEDED with a pointer to the whitepaper and the litmus report.

3a — Structure

- 1. The problem (recomposition cascade, GC storms, FFI marshalling, main-thread blocking)
- 2. The thesis — sharpened, with the canonical ROADMAP's corrected boundary of the claim
- 3. The Triad Protocol — corrected single-atomic-exchange design, with the worked trace from RFC 0001 §4.4
- 4. The frame envelope — Tier 0 frozen
- 5. Writer revocation (I6) — the use-after-free guard
- 6. Measured results — Phase 0 litmus + Phase 1 benchmark, all numbers labeled
- 7. Cross-language consistency — C, Rust, TypeScript all pass the same suite
- 8. Platform honesty — Safari 60 Hz cap (WebKit bug 173434), SAB COOP/COEP, the iOS Canvas vs MTKView split, RN as weakest differentiator
- 9. Non-goals — first-class section, not an appendix
- 10. Open questions — Q1–Q5 from ARCHITECTURE.md, listed not hidden
- 11. References — prior art (Compose graphicsLayer, Reanimated SharedValue, LeakCanary, Apache Arrow, graphics triple buffering)

3b — Errata to the founding spec

The whitepaper's appendix explicitly supersedes:

- Founding spec §5 (the two-variable protocol) — withdrawn per RFC 0001 §3
- Founding spec §9.5 (predicted headline numbers) — replaced by Phase 1 measured numbers
- The Phase 1 implementation report (the Rust Android v0.1 code) — that code used the withdrawn protocol; the corrected code is in Phase 0 of this roadmap

Phase 3 success criterion

- Every number labeled MEASURED (with sandbox label) or PREDICTION (with reasoning)
- Every platform claim carries a source citation
- Cross-language consistency results published
- Founding spec PDF marked superseded

Deliverables

- docs/WHITEPAPER.md — source
- PDF: Weft-Whitepaper-v1.0.pdf (typeset, ACM-style)
- PDF: Weft-Spec-Errata-v0.1.1.pdf (the explicit supersession)

Phase 4 — Source-only platform implementations

Platform ports written as production-ready source, structurally validated, but not compiled in-sandbox (no toolchain). Each port is a Phase 6+ deliverable waiting for the constraint to lift.

4a — Kotlin/Android kernel + Steward + Heddle

- Kernel in pure Kotlin/JVM (no Android dependency; runs on plain JVM if JDK installed)
- Steward with ViewModel integration (source-only; cannot test without AGP)
- Heddle as Modifier.weftDraw extension (source-only; cannot test without Compose runtime)
- JNI bridge to the C kernel (so the Kotlin side calls the litmus-passing C code)
- Structural validator (Python) checks the API surface matches the spec

LOC budget: ~4,700.

4b — Swift/iOS kernel + Steward + Heddle

- Kernel in pure Swift (no Apple framework; would run on Linux via swift-atomics if Swift were installed)
- Steward with @StateObject integration
- Heddle as WeftCanvas (Canvas + CADisplayLink at 60 Hz, MTKView at 120 Hz)
- Structural validator

LOC budget: ~3,500.

4c — Dart/Flutter kernel + Steward + Heddle

- Kernel in pure Dart (no Flutter dependency; would run on Dart VM if dart were installed)
- Steward with StatefulWidget integration
- Heddle as CustomPainter
- Structural validator

LOC budget: ~3,050.

4d — TypeScript Heddles (web framework bindings)

Phase 0e delivers the TS kernel. Phase 4d wraps it for React, Svelte, Vue, and React Native (Reanimated).

- @weft/react — WeftCanvas component, mounts a Worker, transfers OffscreenCanvas
- @weft/svelte — Svelte action equivalent
- @weft/vue — Vue 3 composable
- @weft/react-native — Reanimated SharedValue integration

LOC budget: ~1,500 across four bindings.

Phase 4 success criterion

- Every source file passes its structural validator (API surface matches spec, no missing functions, panic shielding on every JNI entry, SAFETY comments on every unsafe block)
- Every source file has a one-paragraph "why exists" header citing the spec section that justifies it
- Each port is marked STATUS: SOURCE-ONLY, PENDING REAL-DEVICE VERIFICATION in its README

Deliverables

- `core/kotlin/`, `core/swift/`, `core/dart/` — kernel source
- `steward/kotlin/`, `steward/swift/`, `steward/dart/` — steward source
- `heddles/android/`, `heddles/swiftui/`, `heddles/flutter/`, `heddles/react/`, `heddles/svelte/`, `heddles/vue/`, `heddles/react-native/` — Heddle source
- `tools/validate-structure.py` — Python structural validator
- PDF: `Weft-Phase4-Platform-Source-Report.pdf`

Phase 5 — Sandbox release artifact

The sandbox-buildable project is complete at this phase. The deliverable is a single tarball that a contributor with a real dev machine can unpack, build, and use.

5a — Release tarball contents

weft-sandbox-v0.1.tar.gz	
├ README.md	(what's runnable here vs what needs a real device)
├ docs/	(all docs + Phase 3 whitepaper + errata)
├ core/	
│ └ c/	(litmus-passing)
│ └ rust/	(litmus-passing)
│ └ ts/	(litmus-passing)
│ └ kotlin/	(source-only)
│ └ swift/	(source-only)
│ └ dart/	(source-only)
├ steward/	(per-language)
├ heddles/	(per-framework)
├ litmus/	
│ └ catalog.yaml	(language-agnostic)
│ └ runners/	(c, rust, ts)
│ └ REPORT.md	(Phase 0 results)
├ bench/	
│ └ harness/	(Python)
│ └ workloads/	(W1-W5 configs)
│ └ results/	(result bundles)
│ └ site/	(static HTML)
├ tools/	
│ └ weft-probe/	(C + Rust)
│ └ weft-record/	(C + Rust)
├ rfcs/	(RFC 0001 + future)
├ reports/	(all PDFs)
└ INSTALL.md	(how to build on a real dev machine)

5b — Install guide

INSTALL.md explains:

- What runs in-sandbox (C, Rust, TS, Python, tools, litmus, benchmark)
- What needs a real dev machine (Kotlin/Android, Swift/iOS, Dart/Flutter)
- How to install Rust + Node + Python (already in-sandbox; for contributors)
- How to install JDK + Android SDK (for the Kotlin port; not in-sandbox)
- How to install Xcode (for the Swift port; not in-sandbox)
- How to install Flutter SDK (for the Dart port; not in-sandbox)

Phase 5 success criterion

- Tarball builds cleanly when unpacked on a fresh Linux machine with Rust + Node + Python installed
- make litmus runs all 24 test combinations (8 tests × 3 languages) and exits 0
- make bench runs the harness against C and Rust kernels and produces result bundles
- make site regenerates the static benchmark site from result bundles
- make validate runs the structural validator against all source-only platform implementations

Deliverables

- `weft-sandbox-v0.1.tar.gz` — release tarball
- `INSTALL.md` — install guide
- `SHA256SUMS` — checksums
- PDF: `Weft-Phase5-Release-Report.pdf`

Phase 6+ — Real-device verification (DEFERRED)

CONSTRAINT ACKNOWLEDGED

This phase does not start until the sandbox constraint lifts. It is the next phase after Phase 5, but it cannot be executed in-sandbox. The plan is documented here so the transition is mechanical when the constraint lifts.

6a — Compile platform source against the litmus-passing kernels

- Kotlin/Android: build the .aar, link against the C kernel via JNI
- Swift/iOS: build the .xcframework, link against the C kernel via C interop
- Dart/Flutter: build the pub package, link via dart:ffi
- TypeScript/Web: deploy the TS kernel to npm, publish @weft/core

6b — Run the litmus suite on real hardware

- Android: Pixel 7a (mid), Realme C55 (low), Galaxy S24 (high)
- iOS: iPhone SE 2022 (low), iPhone 15 Pro (high)
- Web: Chrome 131, Firefox 132, Safari 17.6 on M2 MacBook Air

Each real-device run produces a result bundle in the same format as the sandbox bundles. The static site renders both side-by-side.

6c — Replace sandbox numbers with real-device numbers

The benchmark site's headline charts switch from x86_64-sandbox to pixel-7a-android (or equivalent). Sandbox numbers remain accessible via a toggle (transparency).

6d — Public launches

- Maven Central: dev.weft:android:0.1.0
- npm: @weft/core, @weft/react, @weft/svelte, @weft/vue, @weft/react-native
- Swift Package: Weft
- pub.dev: weft
- weft.dev: launch the static site publicly
- HN, r/androiddev, r/iOSProgramming, r/flutter, r/webdev: launch posts

6e — Whitepaper v1.1

Replace the x86_64-sandbox numbers in the whitepaper with real-device numbers. Sandbox numbers remain in an appendix (transparency).

Phase 6+ success criterion

- All platform source from Phase 4 compiles cleanly against the litmus-passing kernels
- The litmus suite passes on every real device (L1–L8, all green)

- A stranger can reproduce a published number on their own hardware (the ROADMAP's success criterion for Phase 3)

Endpoint: when is the sandbox-buildable project "done"?

At Phase 5, the sandbox-buildable project is complete:

- Litmus-passing kernel in **three languages** (C, Rust, TypeScript), all eight tests green in all three
- Benchmark harness with measured numbers (clearly labeled x86_64-sandbox)
- Tools (weft-probe, weft-record) in C and Rust
- Whitepaper with real sandbox numbers
- Source-only platform implementations (Kotlin, Swift, Dart) — structurally validated, ready for Phase 6+
- Release tarball with install guide

CONSTRAINT ACKNOWLEDGED

The next phase (real-device verification, Phase 6+) cannot proceed until the sandbox constraint lifts. That is the explicit endpoint. Everything after Phase 5 is gated on the constraint.

What this roadmap does NOT change

- **The Four Laws are still enforced.** Law 2 (zero is a contract) is asserted at 0 B/frame for C and D in the benchmark harness; Law 4 (honesty) now includes "sandbox numbers are labeled sandbox numbers."
- **The litmus suite is still the canonical core.** Three languages pass it; that's three independent proofs of the protocol's soundness.
- **The frame envelope is still Tier 0 frozen.**
- **Writer revocation (I6) is still implemented from day one.**
- **Sequential, benchmark-gated phases.** A phase ends when its success criterion is met.
- **The Pro tier is still out of scope** (separate repo, BSL, charter clause 4).

The roadmap's one rule (unchanged)

A phase without a green success criterion does not get followed by the next phase. Slippage is published.

The sandbox constraint does not relax this rule; it tightens it. A phase that claims green without producing the artifacts (kernel source, litmus results, benchmark bundles, PDFs) has not finished, regardless of how much code was written.